SENIOR ARCHITECT MASTERCLASS

AI Agents & Tool Use

8 Topics across 7 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

Agent Architectures

ReAct, Plan-Execute, LATS, Reflexion

> Imagine you give a new employee a task: 'Book a meeting room for 10 people next Tuesday.' A normal LLM just TALKS about booking -- it cannot actually DO anything. An Agent can think about the task, open the calendar, check room availability, and actually book the room. An Agent = LLM that can THINK + ACT + OBSERVE results + REPEAT until the job is done.

1. ReAct (Reason + Act) | 2. Plan-Execute + LATS + Reflexion

1. ReAct (Reason + Act)

📖 DEEP TEACH

```
Imagine a detective solving a case:
  THINK: 'The victim was last seen at the restaurant. I should check the security cameras.'
  ACT: Goes to the restaurant and checks cameras.
  OBSERVE: 'Camera shows the victim leaving at 9pm with a tall man.'
  THINK: 'I need to identify this man. Let me check the guest list.'
  ACT: Checks the guest list.
  OBSERVE: 'Only one tall man on the list: John Doe.'
  THINK: 'I have enough information now.'
  FINAL ANSWER: 'John Doe was the last person with the victim.'

That is ReAct: Think -> Act -> Observe -> Think -> Act -> Observe -> Answer.

The LLM THINKS about what to do next. Then it picks a TOOL to use (search a database, call an API, run a calculator). Then it
OBSERVES the result. Then it THINKS again. This loop continues until it has the answer.

WHY THE 'THINK' STEP MATTERS:
Without it, the agent randomly tries tools hoping something works. With the Think step, it PLANS each action. Like the detective
reasoning before each move, not just randomly searching buildings.

ReAct is the DEFAULT agent pattern. Use it for 80% of agent tasks.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Imagine giving a new intern their first task. You do not want them running around randomly. You want them to THINK before each
step, DO one thing, LOOK at the result, then THINK about what is next.

1. Set a maximum number of steps (5-10). Without a limit, the agent can loop forever, like a dog chasing its tail.
2. Tool descriptions are the #1 factor. If the detective's notebook says 'magnifying glass: use for everything,' they will try to
use it for everything. Write clear descriptions: 'Calculator: use for math AFTER getting data from search.'
3. STUCK DETECTOR: if the agent calls the same tool 3 times with the same input, force a different approach. Like telling the
detective 'you already searched that building 3 times. Try somewhere else.'
4. Cost: each think-act-observe loop costs one LLM call. 5 loops = $0.15. Budget accordingly.

WAR STORY: An agent was supposed to search a database, then calculate a price. Instead, it called the calculator first (with no
data!) and got stuck in a loop. Fix: improved the tool description to say 'Use AFTER getting numbers from database search.'
```

📋 POCKET NOTES

```
ReAct = Think -> Act -> Observe -> repeat until done
Analogy: detective solving a case step by step
Default for 80% of agent tasks
Max steps: 5-10 (prevents infinite loops)
Cost: ~$0.15 per query (5 steps x $0.03 each)
Tool descriptions = #1 factor for correct tool choice
Stuck detector: same action 3x = force different approach
Log all Think steps for debugging
Upgrade to Plan-Execute for complex 5+ step tasks
```

🎯 INTERVIEW PREP

SAY THIS: 'ReAct is my default agent architecture. It is simple, debuggable, and works for 80% of tasks. I set max iterations, log all reasoning steps, and implement a stuck detector that forces a different approach if the same action repeats.'

Q: When does ReAct fail?
A: 'Complex multi-step tasks where the agent needs to plan ahead. ReAct is reactive -- it figures out the next step after each observation. For tasks needing 5+ coordinated steps, Plan-and-Execute is better because it creates the full plan upfront.'

Q: How do you debug an agent stuck in a loop?
A: 'Read the Think logs. Usually the tool descriptions are ambiguous, the tool returns an unexpected format, or the task is too hard for this architecture.'

⌨ CODE

Python

```python
from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain.tools import tool
from langchain import hub

@tool
def search_db(query: str) -> str:
    """Search company knowledge base for product/policy info. Use FIRST before calculating."""
    db = {"refund":"30-day refund. Electronics: 15 days.",
          "pricing":"Pro: $49/mo. Enterprise: $199/mo. Annual: 20% off."}
    for k,v in db.items():
        if k in query.lower(): return v
    return "No results."

@tool
def calculator(expr: str) -> str:
    """Calculate math. Use AFTER getting numbers from search_db."""
    try: return str(eval(expr))
    except: return "Error"

agent = create_react_agent(ChatOpenAI(model="gpt-4",temperature=0),
    [search_db,calculator], hub.pull("hwchase17/react"))
executor = AgentExecutor(agent=agent,tools=[search_db,calculator],verbose=True,max_iterations=5)
result = executor.invoke({"input":"Pro plan for a year with annual discount?"})
print(f"Answer: {result['output']}")
```

📺 OUTPUT

```
Think: I need the Pro price and annual discount.
Act: search_db('pricing')
Observe: Pro: $49/mo. Annual: 20% off.

Think: Pro is $49/mo. Annual = 12 months. 20% off. Let me calculate.
Act: calculator('49 * 12 * 0.8')
Observe: 470.4

Think: I have the answer.
Answer: $470.40/year with annual discount.

# The agent THOUGHT before each action. Like a detective following clues.
```

2. Plan-Execute + LATS + Reflexion

📖 DEEP TEACH

3 MORE AGENT PATTERNS (for when the detective approach is not enough):

1. PLAN-AND-EXECUTE:
Imagine a project manager who writes the FULL plan before anyone starts working:
  'Step 1: Research competitors. Step 2: Analyze pricing. Step 3: Write report. Step 4: Review.'

Then each step is executed one by one. If something unexpected happens, the PM adjusts the plan.

ReAct = detective figuring it out as they go.
Plan-Execute = project manager with a written plan.

Best for: multi-step tasks like research reports, data analysis, complex workflows.

2. LATS (Language Agent Tree Search):
Imagine a chess player who considers 3 different opening moves, thinks 5 moves ahead for each, and picks the best path.

The agent generates multiple possible next actions, scores each one, and follows the best path. If a path leads to a dead end, it backtracks and tries another.

Sounds powerful but VERY expensive. 20-50 LLM calls per query. Mostly used in research, not production.

3. REFLEXION:
Imagine a student who writes an essay, gets it graded, reads the teacher's comments, and writes a BETTER version. Then gets graded again and improves once more.

The agent attempts the task. An evaluator checks the output. If bad, the agent REFLECTS on what went wrong and tries again with that reflection as extra context.

Best for: code generation (run the code, see errors, fix and retry), writing, analysis.
Max 3 retries. After 3, escalate to a human.

WHICH TO CHOOSE:
Simple tool use -> ReAct (80% of cases)
Complex multi-step -> Plan-Execute
Need the optimal solution -> LATS (expensive)
Need self-improvement -> Reflexion

🏗 ARCHITECT TIPS + WAR STORIES

1. ReAct handles 80% of cases. Do not over-engineer.
2. Plan-Execute: let independent steps run in PARALLEL for speed.
3. LATS: too expensive for production. A 5-step task with 3 branches = 243 possible paths = 20-50 LLM calls = $0.60-1.50 per query.
4. Reflexion: maximum 3 retries. Diminishing returns after that.
5. COMBINE: Plan-Execute for structure + Reflexion for quality checking at each step.
6. All patterns should be implemented as LangGraph state machines in production.

📋 POCKET NOTES

Plan-Execute: write full plan first, then execute. Multi-step tasks.
Analogy: project manager with a written plan vs detective winging it.
LATS: chess player exploring multiple move paths. Expensive. Research only.
Reflexion: write essay -> get graded -> improve -> repeat. Max 3 retries.
Decision: simple=ReAct | multi-step=Plan-Execute | optimal=LATS | self-correct=Reflexion
Cost: ReAct ~$0.15 | Plan-Exec ~$0.30 | LATS ~$1+ | Reflexion ~$0.45
Combine: Plan-Execute structure + Reflexion quality check
All implement as LangGraph state machines

🎯 INTERVIEW PREP

SAY THIS: 'I choose architecture based on task complexity. ReAct for simple tool use. Plan-Execute for coordinated multi-step workflows. Reflexion when output quality justifies multiple attempts.'

Q: LATS sounds powerful. Why not use it in production?
A: 'Too expensive. A 5-step task with 3 possible actions per step means 243 paths to explore. Even with pruning, that is 20-50 LLM calls at $0.03 each = $0.60-1.50 per user query. Plan-Execute at $0.30 is a better tradeoff for 90% of use cases.'

⌨ CODE

Python

```python
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(model="gpt-4",temperature=0)
task = "Total: 3 Pro ($49/mo) + 2 Enterprise ($199/mo), 1 year, 20% discount"

# PLAN first (like a project manager)
plan = (ChatPromptTemplate.from_template("Write a step-by-step plan for: {t}\
Only the plan.")
    | llm | StrOutputParser()).invoke({"t":task})
print("PLAN:",plan[:150])

# EXECUTE each step
steps = [s.strip() for s in plan.split("\
") if s.strip() and s.strip()[0].isdigit()]
results = []
for step in steps:
    r = (ChatPromptTemplate.from_template("Task:{t}\
Step:{s}\
Previous results:{p}\
Execute this step.")
        | llm | StrOutputParser()).invoke({"t":task,"s":step,"p":str(results[-2:])})
    results.append(r)
    print(f"  {step[:40]}... -> {r[:40]}")
print(f"FINAL: {results[-1]}")
```

📺 **OUTPUT**

```
PLAN: 1. Calculate annual cost for 3 Pro plans
2. Calculate annual cost for 2 Enterprise plans
3. Apply 20% discount to both...

  1. Calculate 3 Pro annual... -> 3 x $49 x 12 = $1,764
  2. Calculate 2 Enterprise... -> 2 x $199 x 12 = $4,776
  3. Apply 20% discount... -> $1,411.20 + $3,820.80
  4. Sum total... -> $5,232.00
FINAL: $5,232.00
# Plan written FIRST. Then each step executed in order.
```

Tool / Function Calling

LLM decides, your code executes

> Imagine you have a very smart assistant who can think and talk, but has no hands. They cannot open doors, make phone calls, or type on a computer. Tools give them 'hands.' The assistant DECIDES which tool to use and what to do with it. But YOUR CODE actually does the work. This separation is the safety boundary -- the LLM thinks, you execute.

3. OpenAI + Anthropic + Parallel Calls

📖 DEEP TEACH

```
HOW IT WORKS:
Imagine you tell your assistant: 'Here are tools you can use: a phone, a computer, and a filing cabinet. Tell me WHICH one you want to use and WHAT you want to do with it. I will do it for you.'

The assistant says: 'I want to use the phone to call the restaurant at 555-1234.'
YOU pick up the phone and call. The assistant never touches the phone.

Same with LLMs:
1. You define tools: 'search_database(query)', 'send_email(to, subject, body)', 'calculator(expression)'
2. User asks a question.
3. The LLM says: 'I want to call search_database with query=refund policy'
4. YOUR CODE runs the actual database search.
5. You send the results back to the LLM.
6. The LLM reads the results and writes the final answer.

CRITICAL SAFETY POINT:
The LLM NEVER runs tools itself. It only DECIDES. Your code EXECUTES. This means a clever prompt injection cannot make the LLM directly access your database -- it can only REQUEST access, and your code validates the request.

PARALLEL TOOL CALLS:
Imagine your assistant says: 'I need weather for Tokyo AND Paris.' They can request BOTH at once. GPT-4 and Claude can request multiple tools simultaneously. You run both searches at once, saving time.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of tools like giving a child a wish list. They can WISH for anything, but YOU decide which wishes to grant.

1. TOOL DESCRIPTIONS are everything. If you describe a tool vaguely ('does stuff'), the LLM will use it for everything. Be specific: 'Search company knowledge base for product and policy questions. Returns text. Do NOT use for math.'
2. Keep tool count under 15. More tools = more confusion about which to pick. Like giving someone 50 remote controls instead of one universal remote.
3. VALIDATE all inputs on your server. The LLM might send weird or invalid parameters. Check before executing.
4. SENSITIVE tools (refund, delete account) need a human approval step. Never auto-execute destructive actions.
5. Log every tool call. This is your audit trail.
```

📋 POCKET NOTES

```
LLM decides WHICH tool + WHAT inputs. YOUR code executes. Safety boundary.
Analogy: smart assistant with no hands. They point, you do.
OpenAI: tools=[{type:'function',function:{name,description,parameters}}]
Anthropic: tools=[{name,description,input_schema}]
Parallel: GPT-4/Claude request multiple tools at once
Descriptions: MOST important factor for correct tool selection
Max tools: ~15 per agent (more = confusion)
Validate inputs server-side
Sensitive tools: require human approval
Log every call for audit trail
```

🎯 INTERVIEW PREP

SAY THIS: 'I design tool schemas with the same care as API contracts. The description quality directly determines tool selection accuracy. I keep tools under 15, validate all inputs server-side, and gate sensitive actions behind human approval.'

Q: LLM keeps choosing the wrong tool?
A: 'Five-step diagnosis: better descriptions with explicit when-to-use and when-NOT-to-use, reduce tool overlap by merging similar ones, add system prompt guidance on tool selection, and log selection patterns to find systematic mismatches.'

Q: Security model for tool execution?
A: 'The LLM never executes directly -- it only requests. My code validates every request: parameterized inputs (never raw SQL), server-side type checking, rate limiting, risk-tiered approval gates, and complete audit logging.'

⌨ CODE

Python

```python
from openai import OpenAI
import json
client = OpenAI()

tools = [{
    "type":"function","function":{
        "name":"get_weather",
        "description":"Get current weather for a city. Use for weather questions ONLY.",
        "parameters":{"type":"object","properties":{
            "city":{"type":"string","description":"City name like 'Tokyo'"}},"required":["city"]}}
}]

# LLM DECIDES which tools to call
resp = client.chat.completions.create(
    model="gpt-4",tools=tools,
    messages=[{"role":"user","content":"Weather in Tokyo and Paris?"}])

# YOUR CODE executes the tools
for tc in resp.choices[0].message.tool_calls:
    args = json.loads(tc.function.arguments)
    print(f"LLM requested: {tc.function.name}({args})")
    print(f"YOUR CODE runs: fetching weather for {args.get('city')}...")
    print(f"Result: {args.get('city')}: 22C sunny\
")
```

📺 OUTPUT

```
LLM requested: get_weather({'city': 'Tokyo'})
YOUR CODE runs: fetching weather for Tokyo...
Result: Tokyo: 22C sunny

LLM requested: get_weather({'city': 'Paris'})
YOUR CODE runs: fetching weather for Paris...
Result: Paris: 22C sunny

# TWO parallel requests! LLM decided. YOUR code executed. Safe.
```

Agent Frameworks

LangGraph, CrewAI, AutoGen, smolagents

> Building an agent from scratch is like building a car from raw metal. Frameworks give you pre-built parts: the engine (LLM management), the steering (tool routing), the dashboard (observability). LangGraph is the production standard in 2024-2025.

4. 6 Frameworks Compared

📖 DEEP TEACH

```
6 FRAMEWORKS (know all, master one):

1. LANGGRAPH (THE PRODUCTION STANDARD):
Imagine a flowchart you can actually run. Your agent is a graph with boxes (functions) connected by arrows (transitions). Some
arrows are conditional: 'if billing question, go to billing handler. If technical, go to tech handler.'
Built-in: save progress (if server crashes, resume where you left off), pause for human approval, stream progress to users.
Use for: any production agent. Maximum control.

2. CREWAI (quick multi-agent teams):
Imagine assembling a team: 'You are the researcher. You are the writer. You are the editor.' Each agent has a role, a goal, and a
backstory. Tasks are assigned to specific agents.
Use for: quick prototypes of multi-agent workflows. Think of it as a casting director for AI actors.

3. AUTOGEN (Microsoft, conversation-based):
Imagine agents sitting in a meeting room, discussing a problem. They take turns speaking and building on each other's ideas.
Use for: brainstorming, debate, peer review simulation.

4. SMOLAGENTS (HuggingFace, lightweight):
Imagine a Swiss Army knife instead of a toolbox. Minimal, no overhead. Agents write Python code as their actions.
Use for: simple agents, learning, when you do not want a heavy framework.

5. OPENAI ASSISTANTS API (managed, zero setup):
Imagine hiring a contractor instead of building a team. OpenAI manages everything: code execution, file search, conversation
threads.
Use for: quick demos, when you want zero infrastructure.

6. ANTHROPIC AGENTS (Claude, computer use):
Imagine an AI that can see your screen, move the mouse, and type on the keyboard. Claude's computer use feature can control a
browser like a human.
Use for: complex reasoning, browser automation.

DECISION:
Production (90% of cases) -> LangGraph
Quick multi-agent prototype -> CrewAI
Debate/brainstorming -> AutoGen
Learning/simple -> smolagents
Zero infrastructure -> OpenAI Assistants
Browser control -> Anthropic computer use
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of frameworks like choosing a vehicle. LangGraph = a pickup truck (can do everything, customizable). CrewAI = a shuttle bus
(moves a team quickly). Smolagents = a bicycle (light, fast for one person).

1. LangGraph IS the standard. Learn it deeply. Everything else is secondary.
2. CrewAI is great for demos but limited when you need custom flows, human approval, or state persistence.
3. Do NOT mix frameworks. Pick one and commit. Mixing LangChain + CrewAI + AutoGen = maintenance nightmare.
4. OpenAI Assistants: vendor lock-in but zero infrastructure to manage.
5. All frameworks are converging on the 'graph of nodes' pattern. LangGraph was first.

WAR STORY: Team started with CrewAI for a customer support agent. Worked great for 2 months. Then they needed human approval for
```

refunds, conditional routing by department, and conversation state persistence. Migrated to LangGraph in 2 days. Every requirement mapped to a built-in feature. Lesson: start with LangGraph if you expect any complexity.

📋 POCKET NOTES

```
LangGraph: flowchart you can run. PRODUCTION STANDARD.
  Conditional routing, save progress, human approval, streaming.
CrewAI: cast AI actors with roles. Quick multi-agent prototypes.
AutoGen: agents in a meeting room discussing. Debate/brainstorm.
smolagents: Swiss Army knife. Lightweight. Learning/simple.
Assistants API: hire a contractor. Zero infra. Demos.
Anthropic: Claude sees your screen. Browser control.
Decision: production=LangGraph(90%) | team proto=CrewAI | managed=Assistants
Do NOT mix frameworks. Pick one, commit.
```

🎯 INTERVIEW PREP

SAY THIS: 'LangGraph is my production framework. It gives maximum control with conditional routing, state persistence, and human-in-the-loop. For quick multi-agent prototypes, CrewAI. I evaluate frameworks on production readiness, debugging tools, and community size.'

Q: LangGraph vs CrewAI?
A: 'LangGraph for production: custom flow control, human approval gates, state that survives server restarts. CrewAI for prototypes: quick role-based multi-agent setups. If you will ever need the production features, start with LangGraph.'

Q: OpenAI Assistants API?
A: 'Good for demos and simple apps. For production: vendor lock-in, limited customization, cannot inspect intermediate state, single-provider dependency. I prefer building on LangGraph with the ability to swap providers.'

⌨️ CODE

Python

```python
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4",temperature=0)

class State(TypedDict):
    query: str
    category: str
    answer: str

def classify(state):
    r = llm.invoke(f"Classify billing/technical/general: {state['query']}. One word.")
    state["category"] = r.content.strip().lower()
    return state

def billing(s): s["answer"]=f"[Billing] Pro=$49/mo. Re: {s['query']}"; return s
def technical(s): s["answer"]=f"[Tech] Troubleshooting: {s['query']}"; return s
def general(s): s["answer"]=f"[General] {s['query']}"; return s
def route(s): return s["category"]

g = StateGraph(State)
for name,fn in [("classify",classify),("billing",billing),("technical",technical),("general",general)]:
    g.add_node(name,fn)
g.set_entry_point("classify")
g.add_conditional_edges("classify",route)
for n in ["billing","technical","general"]: g.add_edge(n,END)
app = g.compile()

for q in ["Pro plan price?","API 500 error","Hello!"]:
    r = app.invoke({"query":q,"category":"","answer":""})
    print(f"{q} -> {r['answer']}")
```

📺 OUTPUT

```
Pro plan price? -> [Billing] Pro=$49/mo. Re: Pro plan price?
API 500 error -> [Tech] Troubleshooting: API 500 error
Hello! -> [General] Hello!

# Queries routed via conditional edges in the graph.
# Like a flowchart: classify -> route -> handle -> done.
```

Multi-Agent Systems

Teams of AI agents working together

> Imagine one person trying to research a topic, write an article, AND edit it for grammar. They will do a mediocre job at all three. Now imagine a TEAM: a researcher, a writer, and an editor. Each person is excellent at their ONE job. Multi-agent = giving AI agents specialized roles and having them collaborate.

5. 4 Multi-Agent Patterns

📖 DEEP TEACH

```
4 WAYS TO ORGANIZE AN AI TEAM:

1. ORCHESTRATOR-WORKER (boss + specialists):
Imagine a project manager who says: 'Researcher, find data. Writer, create the draft. Editor, polish it.' The manager coordinates
but does not do the actual work.
This is the most common pattern in production.

2. DEBATE (argue and decide):
Imagine two lawyers in court. One argues FOR using PostgreSQL. The other argues FOR MongoDB. A judge listens to both and decides.
Great for: reducing bias in AI decisions. When one agent always says 'yes,' the other agent catches problems.

3. DELEGATION (chain of command):
Imagine a corporate hierarchy: CEO tells VP, VP tells manager, manager tells developer. Each level handles their scope.
Use for: complex enterprise workflows with clear authority levels.

4. PEER REVIEW (check each other's work):
Imagine a code review process. Agent 1 writes code. Agent 2 reviews it and finds bugs. Agent 1 fixes the bugs.
Use for: quality-critical outputs (code, legal documents, medical reports).

CRITICAL RULE: KEEP TEAMS SMALL.
2-4 agents is ideal. Each agent makes 3-5 LLM calls. A team of 4 = about 15-20 LLM calls per task. A team of 7 = 35+ calls = $2 per
query and 45 seconds of waiting.

More agents does NOT mean better results. The coordination overhead (agents talking to each other) grows faster than the benefit.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like a startup. A 3-person startup (CEO, engineer, designer) ships fast. A 20-person committee has endless meetings and
ships nothing.

1. Orchestrator-Worker is the most common production pattern.
2. Debate is UNDERRATED for reducing AI bias. When one agent always agrees, have another play devil's advocate.
3. Role descriptions matter a LOT. '10-year senior data analyst who always cites numbers' produces much better output than
'analyst.'
4. Shared memory: agents need a common scratchpad to pass context between each other.

WAR STORY: Team built a 7-agent 'company simulation.' It cost $2 per query and took 45 seconds. They reduced to 3 agents
(researcher, writer, reviewer) and got 90% of the quality at 20% of the cost. The 4 extra agents mostly repeated what the first 3
already said.
```

📋 POCKET NOTES

```
4 Patterns:
  Orchestrator-Worker: boss delegates to specialists (MOST COMMON)
  Debate: 2 argue, judge decides (reduces AI bias)
  Delegation: CEO -> VP -> Worker chain of command
  Peer Review: agents check each other's work (quality-critical)
Team size: 2-4 agents. More = cost explosion + coordination overhead.
Each agent: 3-5 LLM calls. Team of 4 = ~15-20 calls per task.
Role descriptions: specific > vague ('10yr analyst' > 'analyst')
Debate: underrated for reducing AI decision bias
```

🎯 INTERVIEW PREP

SAY THIS: 'I design multi-agent teams with the minimum viable headcount -- 2-4 agents. Orchestrator-worker by default. Debate pattern when I need to reduce AI decision bias.'

Q: When is multi-agent overkill?
A: 'When a single agent with good tools can do the job. If subtasks are independent, parallel tool calls within ONE agent are simpler and cheaper than multiple agents. Multi-agent adds value when tasks require different EXPERTISE or when review genuinely improves quality.'

Q: How do you prevent agents from contradicting each other?
A: 'Shared state with clear ownership. Each agent reads from shared context but writes only to their section. The orchestrator merges and resolves conflicts.'

⌨️ CODE

Python

```python
from crewai import Agent, Task, Crew, Process

# A small team: researcher + writer (2 agents, not 7)
researcher = Agent(
    role="Senior Research Analyst",
    goal="Find accurate data with numbers and citations",
    backstory="10-year veteran who never guesses. Always cites sources.",
    verbose=True, allow_delegation=False)

writer = Agent(
    role="Tech Content Writer",
    goal="Write engaging posts from research",
    backstory="Award-winning writer who makes complex topics simple.",
    verbose=True, allow_delegation=False)

crew = Crew(
    agents=[researcher, writer],
    tasks=[
        Task(description="Research AI agents 2025. Find 5 trends with data.",
             expected_output="Research brief with numbers.", agent=researcher),
        Task(description="Write 200-word blog from the research.",
             expected_output="Polished blog post.", agent=writer)],
    process=Process.sequential, verbose=True)

result = crew.kickoff()
print(f"Result: {str(result)[:150]}...")
```

📺 OUTPUT

```
[Researcher] Searching for AI agent trends...
[Researcher] Found 5 trends with market data.
[Writer] Drafting from research brief...
[Writer] 210-word post complete.

Result: AI Agents in 2025: The autonomous revolution has arrived...
# 2 agents, 2 roles, sequential collaboration. Clean and efficient.
```

Agent Memory

Short-term, long-term, entity, episodic

> Imagine talking to someone with amnesia. Every time you say something, they forget what you said 5 seconds ago. Frustrating, right? Without memory, an AI agent is like that. Every message is the first message. Memory lets agents remember your conversation, learn your preferences, and recall past interactions.

6. 4 Memory Types

📖 DEEP TEACH

```
4 TYPES OF MEMORY (like a human brain):

1. SHORT-TERM (what you just said):
Imagine your working memory -- what you are thinking about RIGHT NOW. The agent remembers the last 10 messages in the current conversation.
When message 11 arrives, message 1 is forgotten (like how you forget what you had for lunch last Tuesday).
ALWAYS use this. It is the bare minimum.

2. LONG-TERM (what you discussed weeks ago):
Imagine a journal. You write down what happened each day. Months later, you can search: 'What did I discuss with this customer last March?'
The agent summarizes old conversations and stores them. When needed, it searches for relevant past conversations.

3. ENTITY MEMORY (facts about people and things):
Imagine a personal CRM that auto-updates. After chatting, the agent notes:
  'Alice: CTO at Acme, prefers Python, uses VS Code, last called March 15.'
Next time Alice calls, the agent already knows her preferences. Auto-extracted from conversations.

4. EPISODIC MEMORY (lessons learned):
Imagine experience. 'Last time we tried approach X, it failed because Y. Do not repeat that mistake.'
The agent remembers specific events and their outcomes, and uses those lessons for future decisions.

THE SPACE PROBLEM:
Memory competes with everything else for the LLM's limited attention span:
  System instructions (1000 words) + Tools (1000 words) + Memory (3000 words) + User question (1000 words) + Space for answer (2000 words) = 8000 words total.

Solution: SUMMARIZE old conversations. Instead of keeping 50 messages, compress them into 3 sentences. Same knowledge, 1/50th the space.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like packing a suitcase. You have limited space. You cannot bring every outfit you own. You bring: what you need today (short-term), a summary of your plans (long-term summary), and your essential info (entity facts).

1. Most agents only need short-term memory. Start simple.
2. Entity memory is amazing for personalization. After each conversation, extract key facts automatically.
3. Summarization saves tokens. 50 messages compressed to 3 sentences = same knowledge, 50x less space.
4. Add long-term memory only when users say 'remember what we discussed last time.' Do not over-engineer.
5. Production stack: short-term (always) + entity memory (if you want personalization).
```

📋 POCKET NOTES

```
Short-term: last 10 messages. Always use. Oldest messages drop off.
Long-term: all past conversations summarized/searchable.
Entity: facts about people/things. Auto-CRM.
Episodic: past events + lessons learned.
Analogy: working memory, journal, personal CRM, experience
Space budget: instructions(1K) + tools(1K) + memory(3K) + query(1K) + answer(2K) = 8K
Summarize: 50 messages -> 3 sentences (50x space savings)
Production: short-term (always) + entity (if personalization)
```

🎯 **INTERVIEW PREP**

SAY THIS: 'I implement tiered memory. Short-term buffer always on. Entity memory for personalization. Long-term vector store for cross-session recall. The critical constraint is context window budget -- I plan token allocation across instructions, tools, memory, and generation.'

Q: How to handle memory in an 8K context window?
A: 'Budget: 1K for instructions, 1K for tools, 3K for memory (last 5 messages verbatim, older ones summarized, plus key entity facts), 1K for user query, 2K for answer generation.'

⌨️ **CODE**

Python

```python
from langchain.memory import ConversationBufferWindowMemory
from langchain_openai import ChatOpenAI
import json

# Short-term: keep only last 3 messages
buffer = ConversationBufferWindowMemory(k=3)
buffer.save_context({"input":"I am Alice"},{"output":"Hi Alice!"})
buffer.save_context({"input":"I work at Acme"},{"output":"Cool!"})
buffer.save_context({"input":"I like Python"},{"output":"Great!"})
buffer.save_context({"input":"What is RAG?"},{"output":"RAG is..."})
print("SHORT-TERM (last 3 only):")
print(buffer.load_memory_variables({})["history"][:80])
print("# 'I am Alice' was forgotten -- only 3 messages kept\
")

# Entity memory: extract facts automatically
llm = ChatOpenAI(model="gpt-4")
msg = "I am Alice, CTO at Acme. I prefer Python and VS Code."
resp = llm.invoke(f"Extract key facts as JSON: {msg}")
try:
    entities = json.loads(resp.content)
    print(f"ENTITY MEMORY: {entities}")
    print("# Next time Alice calls, we already know her preferences.")
except: print("Extraction done")
```

📺 **OUTPUT**

```
SHORT-TERM (last 3 only):
Human: I work at Acme
AI: Cool!
Human: I like Python
AI: Great!
# 'I am Alice' was forgotten -- only 3 messages kept

ENTITY MEMORY: {"name":"Alice","role":"CTO","company":"Acme","lang":"Python","editor":"VS Code"}
# Next time Alice calls, we already know her preferences.
```

Human-in-the-Loop

Approval gates and escalation

> Imagine a self-driving car. It drives itself on highways, but when it reaches a confusing intersection, it STOPS and asks the human driver to take over. That is Human-in-the-Loop (HITL). The agent handles routine tasks automatically but PAUSES for human approval on risky or important decisions.

7. Risk-Tiered Approvals

📖 DEEP TEACH

```
3 PATTERNS:

1. APPROVAL GATE (pause and ask permission):
Imagine an assistant who says: 'I want to process a $500 refund for order #123. Shall I proceed?'
The human reviews and says 'Approved' or 'Denied.'

Classify every action by RISK:
  AUTO (safe): searching, calculating, reading data.
  APPROVAL NEEDED (risky): refunding money, sending emails, modifying accounts.
  PERMANENTLY BLOCKED (dangerous): deploying to production, deleting accounts, admin actions.

2. ESCALATION (agent knows its limits):
Imagine a customer service rep who says 'This is above my pay grade. Let me get my manager.'
The agent recognizes: 'I am not confident about this' or 'This topic is sensitive' and passes to a human.

3. INTERACTIVE (human guides step by step):
Imagine a GPS that says: 'I found 3 routes. Route A is fastest. Route B avoids highways. Route C is most scenic. Which do you prefer?'
The agent presents options. The human picks. The agent continues.

LANGGRAPH IMPLEMENTATION:
interrupt_before=['approval_node']
The graph PAUSES before that node. State is saved. Even if the server restarts, it remembers where it stopped. Human approves. Graph resumes.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like employee authority levels. A junior employee can order office supplies (auto). A manager approves team hires (approval). Only the CEO can sign M&A deals (escalation to top).

1. Classify ALL agent actions into risk tiers at design time. Not at runtime.
2. Timeout: if the human does not respond in 24 hours, auto-reject or auto-escalate.
3. SHOW CONTEXT: do not just say 'approve?' Show WHAT the agent wants to do and WHY (its reasoning).
4. Approval fatigue: if humans approve 98% of requests, too many low-risk items need approval. Auto-approve those. Target 70-80% approval rate for meaningful review.
5. Slack/email notifications for pending approvals.
```

📋 POCKET NOTES

```
3 Patterns: Approval Gate | Escalation | Interactive
Risk tiers: AUTO(search) | APPROVAL(refund) | BLOCKED(deploy)
Analogy: employee authority levels (junior/manager/CEO)
LangGraph: interrupt_before=['node'] + checkpointing
Timeout: 24h -> auto-reject or escalate
Show: WHAT agent wants + WHY (reasoning)
Approval fatigue: if approve > 98%, auto-approve more. Target 70-80%.
Notify: Slack/email for pending approvals
```

🎯 INTERVIEW PREP

SAY THIS: 'I use risk-tiered approval gates. Auto-approve reads, human approval for writes, permanent block for admin actions. LangGraph interrupt_before with state checkpointing so the flow survives server restarts.'

Q: How to prevent approval fatigue?
A: 'Batch similar approvals, auto-approve based on learned patterns, show rich context for quick decisions, and target 70-80% approval rate. If humans approve 98%, too many trivial items are in the queue.'

⌨️ CODE

Python

```python
class ApprovalGate:
    TIERS = {"search":"auto","calculate":"auto","read_data":"auto",
             "send_email":"approval","process_refund":"approval",
             "delete_account":"blocked","deploy":"blocked"}

    def check(self, action, details):
        risk = self.TIERS.get(action, "approval")
        if risk == "auto":
            print(f"  AUTO-APPROVED: {action}"); return True
        elif risk == "blocked":
            print(f"  PERMANENTLY BLOCKED: {action}"); return False
        else:
            print(f"  NEEDS HUMAN APPROVAL: {action} -- {details}")
            return True  # simulated approval

gate = ApprovalGate()
print("AGENT ACTION PIPELINE:")
for action, details in [("search",{"q":"policy"}),("process_refund",{"amount":499}),
                        ("delete_account",{"user":"456"}),("calculate",{"expr":"1+1"})]:
    ok = gate.check(action, details)
    print(f"  -> {'Executing' if ok else 'Skipped'}\
")
```

📺 OUTPUT

```
AGENT ACTION PIPELINE:
  AUTO-APPROVED: search
  -> Executing

  NEEDS HUMAN APPROVAL: process_refund -- {'amount': 499}
  -> Executing

  PERMANENTLY BLOCKED: delete_account
  -> Skipped

  AUTO-APPROVED: calculate
  -> Executing

# Safe actions auto-approved. Risky ones need human. Dangerous ones blocked.
```

Code Execution + Browser

E2B sandbox, Playwright, computer use

> Imagine giving an AI a computer and saying 'write code to analyze this data, run it, and show me the results.' The AI writes the code, but you would NOT let it run the code on YOUR computer (it might delete files!). Instead, you give it a disposable computer in a sandbox. If it messes up, you just throw away the sandbox. No damage done.

8. E2B Sandbox + Browser Control

📖 DEEP TEACH

```
CODE EXECUTION:
Imagine giving a child a chemistry set. You would not let them experiment in your kitchen. You would set up a separate table with
protective covers, where they can mix anything without damaging your home.

E2B is that protective table for AI code. It is a cloud sandbox -- an isolated mini-computer that:
- Runs any code the AI writes (Python, JavaScript, anything)
- Cannot access your real files, databases, or servers
- Auto-destroys after 30 seconds if the code runs too long
- Costs $0.10/hour

The AI writes code -> E2B runs it safely -> AI sees the output -> AI fixes and retries if needed (max 3 tries).

NEVER run LLM-generated code on your real servers. ALWAYS use a sandbox.

BROWSER AGENTS:
Imagine an AI that can see a webpage (as a screenshot), decide what to click, and a tool (Playwright) clicks it. Then the AI sees
the new page, decides the next click, and so on.

Two approaches:
1. PLAYWRIGHT (you tell it WHERE things are):
You code the selectors: 'click the button with id=submit'. More reliable but requires knowing the page structure.

2. VISION-BASED (AI figures out WHERE things are):
The AI looks at a screenshot and says 'click at pixel 340, 220.' Works on ANY website without knowing the structure. Less reliable
but more flexible.

Anthropic's 'computer use' lets Claude literally see your screen and control mouse/keyboard.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like a test kitchen for a restaurant. Chefs experiment with new recipes in the test kitchen (sandbox). Only proven
recipes go to the real kitchen (production).

1. E2B: working in 5 minutes. pip install e2b-code-interpreter.
2. NEVER let AI code touch your production systems. No exceptions.
3. 30-second timeout per execution. LLMs sometimes generate infinite loops.
4. Let the AI retry up to 3 times. It sees the error message and often fixes its own code.
5. Pre-install common libraries in the sandbox (pandas, numpy, matplotlib).
6. Browser agents cost about $0.50 per 10-step task (screenshots are expensive to process).
7. Use Playwright for websites you control. Vision-based for unknown websites.
```

📋 POCKET NOTES

```
Code Execution: AI writes code -> sandbox runs it -> AI sees output -> retries if error
E2B: cloud sandbox. $0.10/hr. Isolated. Auto-destroys. pip install e2b-code-interpreter.
NEVER run AI code on your server. Always sandbox.
Timeout: 30 seconds. Max retries: 3.
Browser: screenshot -> AI decides action -> Playwright clicks -> new screenshot -> repeat
Playwright: you define selectors (known sites). Vision: AI reads screenshots (any site).
Anthropic computer use: Claude controls mouse/keyboard.
```

Cost: code ~$0.10/hr | browser ~$0.50 per 10 steps
Security: no network access, resource limits, no credentials in sandbox

🎯 INTERVIEW PREP

SAY THIS: 'For code execution, I use E2B sandboxes -- isolated cloud environments where AI-generated code runs safely without access to production systems. The AI iterates: write code, see output or error, fix, retry up to 3 times.'

Q: How do you secure code execution?
A: 'Isolated sandbox with no network access by default, resource limits (1 CPU, 512MB RAM, 30-second timeout), no production credentials, whitelist-only network access (allow PyPI only), and complete audit logging of all executed code.'

Q: Playwright vs Anthropic computer use?
A: 'Playwright: fast, reliable, cheap, but requires knowing the page structure. Computer use: flexible, works on any UI, but slower, more expensive, less reliable. I use Playwright for known sites and fall back to vision for dynamic or unknown pages.'

⌨️ CODE

Python

```python
from e2b_code_interpreter import Sandbox
from openai import OpenAI

client = OpenAI()
sandbox = Sandbox()  # Isolated cloud computer

task = "Calculate first 10 Fibonacci numbers and their sum"
code = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role":"system","content":"Write Python code. Print results. Code only."},
              {"role":"user","content":task}]
).choices[0].message.content.strip()

# Clean markdown fences if present
lines = code.split("\
")
if lines[0].startswith("python"): lines = lines[1:]
code = "\
".join(lines)
print(f"AI WROTE:\
{code}\
")

# Run in sandbox (NOT on your server!)
exec_result = sandbox.run_code(code)
print("SANDBOX OUTPUT:")
for log in exec_result.logs.stdout: print(f"  {log}")
sandbox.kill()  # Destroy sandbox when done
```

📺 OUTPUT

```
AI WROTE:
def fib(n):
    fibs = [0, 1]
    for i in range(2, n):
        fibs.append(fibs[-1] + fibs[-2])
    return fibs

result = fib(10)
print(f'Fibonacci: {result}')
print(f'Sum: {sum(result)}')

SANDBOX OUTPUT:
  Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
  Sum: 88

# AI wrote the code. Sandbox ran it safely. Your server was never at risk.
```

LIBRARIES

langgraph	Agent state machines (production)	`pip install langgraph`
crewai	Multi-agent role teams	`pip install crewai`
autogen	Conversational agents	`pip install autogen`
smolagents	Lightweight agents	`pip install smolagents`
e2b	Code sandbox	`pip install e2b-code-interpreter`
playwright	Browser automation	`pip install playwright`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```
graph TB
    User([User Message]) --> Classify[Classify Intent<br/>billing / technical / general]

    Classify -->|Billing| BillingAgent
    Classify -->|Technical| TechAgent
    Classify -->|General| GeneralAgent

    subgraph BillingAgent[Billing Handler]
        B_Search[Search Pricing DB] --> B_Generate[Generate Response]
    end

    subgraph TechAgent[Technical Handler]
        T_Search[Search Docs + FAQs] --> T_Generate[Generate Response]
    end

    subgraph GeneralAgent[General Handler]
        G_Generate[Generate Response]
    end

    B_Generate --> RiskCheck
    T_Generate --> RiskCheck
    G_Generate --> RiskCheck

    RiskCheck{Risk Assessment}

    RiskCheck -->|LOW: search, info| AutoApprove[Auto-Execute<br/>No human needed]
    RiskCheck -->|MEDIUM: refund, email| HumanGate[Human Approval Gate<br/>Slack notification]
    RiskCheck -->|HIGH: delete, admin| Block[Permanently Blocked<br/>Cannot proceed]

    HumanGate -->|Approved| Execute
    HumanGate -->|Denied| Deny[Inform User:<br/>Action not approved]
    HumanGate -->|Timeout 24h| Escalate[Escalate to Manager]

    AutoApprove --> Execute[Execute Action]
    Execute --> QualityCheck[Quality Check<br/>LLM scores response 1-10]

    QualityCheck -->|Score >= 7| Respond([Send to User])
    QualityCheck -->|Score < 7| Regenerate[Regenerate Response<br/>Max 2 retries]
    Regenerate --> QualityCheck

    Block --> AuditLog
    Deny --> AuditLog
    Respond --> AuditLog[Audit Log<br/>Every action + decision + reasoning]

    subgraph Checkpointing
        State[(LangGraph State<br/>SQLite dev / Postgres prod)]
    end

    HumanGate -.-> State
    Execute -.-> State

    style Classify fill:#8957e5,color:#fff
    style RiskCheck fill:#d29922,color:#fff
    style AutoApprove fill:#3fb950,color:#fff
    style HumanGate fill:#d29922,color:#fff
    style Block fill:#da3633,color:#fff
    style QualityCheck fill:#1f6feb,color:#fff
    style AuditLog fill:#3fb950,color:#fff
```

— POCKET NOTES —

1. ReAct (Reason + Act)

```
ReAct = Think -> Act -> Observe -> repeat until done
Analogy: detective solving a case step by step
Default for 80% of agent tasks
```

```
Max steps: 5-10 (prevents infinite loops)
Cost: ~$0.15 per query (5 steps x $0.03 each)
Tool descriptions = #1 factor for correct tool choice
Stuck detector: same action 3x = force different approach
Log all Think steps for debugging
Upgrade to Plan-Execute for complex 5+ step tasks
```

2. Plan-Execute + LATS + Reflexion

```
Plan-Execute: write full plan first, then execute. Multi-step tasks.
Analogy: project manager with a written plan vs detective winging it.
LATS: chess player exploring multiple move paths. Expensive. Research only.
Reflexion: write essay -> get graded -> improve -> repeat. Max 3 retries.
Decision: simple=ReAct | multi-step=Plan-Execute | optimal=LATS | self-correct=Reflexion
Cost: ReAct ~$0.15 | Plan-Exec ~$0.30 | LATS ~$1+ | Reflexion ~$0.45
Combine: Plan-Execute structure + Reflexion quality check
All implement as LangGraph state machines
```

3. OpenAI + Anthropic + Parallel Calls

```
LLM decides WHICH tool + WHAT inputs. YOUR code executes. Safety boundary.
Analogy: smart assistant with no hands. They point, you do.
OpenAI: tools=[{type:'function',function:{name,description,parameters}}]
Anthropic: tools=[{name,description,input_schema}]
Parallel: GPT-4/Claude request multiple tools at once
Descriptions: MOST important factor for correct tool selection
Max tools: ~15 per agent (more = confusion)
Validate inputs server-side
Sensitive tools: require human approval
Log every call for audit trail
```

4. 6 Frameworks Compared

```
LangGraph: flowchart you can run. PRODUCTION STANDARD.
  Conditional routing, save progress, human approval, streaming.
CrewAI: cast AI actors with roles. Quick multi-agent prototypes.
AutoGen: agents in a meeting room discussing. Debate/brainstorm.
smolagents: Swiss Army knife. Lightweight. Learning/simple.
Assistants API: hire a contractor. Zero infra. Demos.
Anthropic: Claude sees your screen. Browser control.
Decision: production=LangGraph(90%) | team proto=CrewAI | managed=Assistants
Do NOT mix frameworks. Pick one, commit.
```

5. 4 Multi-Agent Patterns

```
4 Patterns:
  Orchestrator-Worker: boss delegates to specialists (MOST COMMON)
  Debate: 2 argue, judge decides (reduces AI bias)
  Delegation: CEO -> VP -> Worker chain of command
  Peer Review: agents check each other's work (quality-critical)
Team size: 2-4 agents. More = cost explosion + coordination overhead.
Each agent: 3-5 LLM calls. Team of 4 = ~15-20 calls per task.
Role descriptions: specific > vague ('10yr analyst' > 'analyst')
Debate: underrated for reducing AI decision bias
```

6. 4 Memory Types

```
Short-term: last 10 messages. Always use. Oldest messages drop off.
Long-term: all past conversations summarized/searchable.
Entity: facts about people/things. Auto-CRM.
Episodic: past events + lessons learned.
Analogy: working memory, journal, personal CRM, experience
Space budget: instructions(1K) + tools(1K) + memory(3K) + query(1K) + answer(2K) = 8K
```

```
Summarize: 50 messages -> 3 sentences (50x space savings)
Production: short-term (always) + entity (if personalization)
```

7. Risk-Tiered Approvals

```
3 Patterns: Approval Gate | Escalation | Interactive
Risk tiers: AUTO(search) | APPROVAL(refund) | BLOCKED(deploy)
Analogy: employee authority levels (junior/manager/CEO)
LangGraph: interrupt_before=['node'] + checkpointing
Timeout: 24h -> auto-reject or escalate
Show: WHAT agent wants + WHY (reasoning)
Approval fatigue: if approve > 98%, auto-approve more. Target 70-80%.
Notify: Slack/email for pending approvals
```

8. E2B Sandbox + Browser Control

```
Code Execution: AI writes code -> sandbox runs it -> AI sees output -> retries if error
E2B: cloud sandbox. $0.10/hr. Isolated. Auto-destroys. pip install e2b-code-interpreter.
NEVER run AI code on your server. Always sandbox.
Timeout: 30 seconds. Max retries: 3.
Browser: screenshot -> AI decides action -> Playwright clicks -> new screenshot -> repeat
Playwright: you define selectors (known sites). Vision: AI reads screenshots (any site).
Anthropic computer use: Claude controls mouse/keyboard.
Cost: code ~$0.10/hr | browser ~$0.50 per 10 steps
Security: no network access, resource limits, no credentials in sandbox
```

— INTERVIEW PREP —

1. ReAct (Reason + Act)

```
SAY THIS: 'ReAct is my default agent architecture. It is simple, debuggable, and works for 80% of tasks. I set max iterations, log all
reasoning steps, and implement a stuck detector that forces a different approach if the same action repeats.'

Q: When does ReAct fail?
A: 'Complex multi-step tasks where the agent needs to plan ahead. ReAct is reactive -- it figures out the next step after each
observation. For tasks needing 5+ coordinated steps, Plan-and-Execute is better because it creates the full plan upfront.'

Q: How do you debug an agent stuck in a loop?
A: 'Read the Think logs. Usually the tool descriptions are ambiguous, the tool returns an unexpected format, or the task is too hard for
this architecture.'
```

2. Plan-Execute + LATS + Reflexion

```
SAY THIS: 'I choose architecture based on task complexity. ReAct for simple tool use. Plan-Execute for coordinated multi-step workflows.
Reflexion when output quality justifies multiple attempts.'

Q: LATS sounds powerful. Why not use it in production?
A: 'Too expensive. A 5-step task with 3 possible actions per step means 243 paths to explore. Even with pruning, that is 20-50 LLM calls
at $0.03 each = $0.60-1.50 per user query. Plan-Execute at $0.30 is a better tradeoff for 90% of use cases.'
```

3. OpenAI + Anthropic + Parallel Calls

```
SAY THIS: 'I design tool schemas with the same care as API contracts. The description quality directly determines tool selection
accuracy. I keep tools under 15, validate all inputs server-side, and gate sensitive actions behind human approval.'

Q: LLM keeps choosing the wrong tool?
A: 'Five-step diagnosis: better descriptions with explicit when-to-use and when-NOT-to-use, reduce tool overlap by merging similar ones,
add system prompt guidance on tool selection, and log selection patterns to find systematic mismatches.'

Q: Security model for tool execution?
```

A: 'The LLM never executes directly -- it only requests. My code validates every request: parameterized inputs (never raw SQL), server-side type checking, rate limiting, risk-tiered approval gates, and complete audit logging.'

4. 6 Frameworks Compared

SAY THIS: 'LangGraph is my production framework. It gives maximum control with conditional routing, state persistence, and human-in-the-loop. For quick multi-agent prototypes, CrewAI. I evaluate frameworks on production readiness, debugging tools, and community size.'

Q: LangGraph vs CrewAI?
A: 'LangGraph for production: custom flow control, human approval gates, state that survives server restarts. CrewAI for prototypes: quick role-based multi-agent setups. If you will ever need the production features, start with LangGraph.'

Q: OpenAI Assistants API?
A: 'Good for demos and simple apps. For production: vendor lock-in, limited customization, cannot inspect intermediate state, single-provider dependency. I prefer building on LangGraph with the ability to swap providers.'

5. 4 Multi-Agent Patterns

SAY THIS: 'I design multi-agent teams with the minimum viable headcount -- 2-4 agents. Orchestrator-worker by default. Debate pattern when I need to reduce AI decision bias.'

Q: When is multi-agent overkill?
A: 'When a single agent with good tools can do the job. If subtasks are independent, parallel tool calls within ONE agent are simpler and cheaper than multiple agents. Multi-agent adds value when tasks require different EXPERTISE or when review genuinely improves quality.'

Q: How do you prevent agents from contradicting each other?
A: 'Shared state with clear ownership. Each agent reads from shared context but writes only to their section. The orchestrator merges and resolves conflicts.'

6. 4 Memory Types

SAY THIS: 'I implement tiered memory. Short-term buffer always on. Entity memory for personalization. Long-term vector store for cross-session recall. The critical constraint is context window budget -- I plan token allocation across instructions, tools, memory, and generation.'

Q: How to handle memory in an 8K context window?
A: 'Budget: 1K for instructions, 1K for tools, 3K for memory (last 5 messages verbatim, older ones summarized, plus key entity facts), 1K for user query, 2K for answer generation.'

7. Risk-Tiered Approvals

SAY THIS: 'I use risk-tiered approval gates. Auto-approve reads, human approval for writes, permanent block for admin actions. LangGraph interrupt_before with state checkpointing so the flow survives server restarts.'

Q: How to prevent approval fatigue?
A: 'Batch similar approvals, auto-approve based on learned patterns, show rich context for quick decisions, and target 70-80% approval rate. If humans approve 98%, too many trivial items are in the queue.'

8. E2B Sandbox + Browser Control

SAY THIS: 'For code execution, I use E2B sandboxes -- isolated cloud environments where AI-generated code runs safely without access to production systems. The AI iterates: write code, see output or error, fix, retry up to 3 times.'

Q: How do you secure code execution?
A: 'Isolated sandbox with no network access by default, resource limits (1 CPU, 512MB RAM, 30-second timeout), no production credentials, whitelist-only network access (allow PyPI only), and complete audit logging of all executed code.'

Q: Playwright vs Anthropic computer use?
A: 'Playwright: fast, reliable, cheap, but requires knowing the page structure. Computer use: flexible, works on any UI, but slower, more expensive, less reliable. I use Playwright for known sites and fall back to vision for dynamic or unknown pages.'